



CSE 4513

Lec - 8

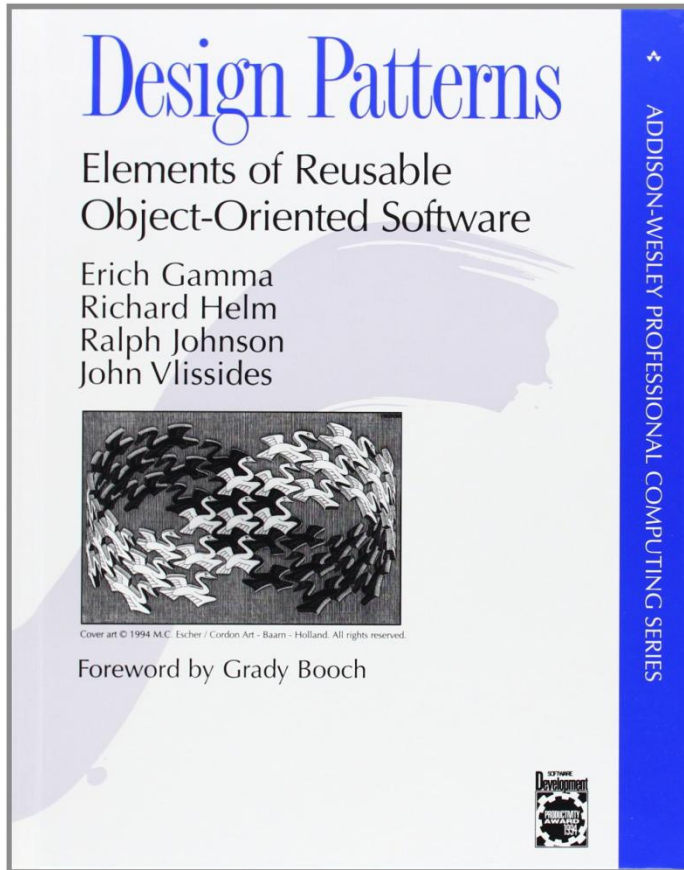
Design Pattern



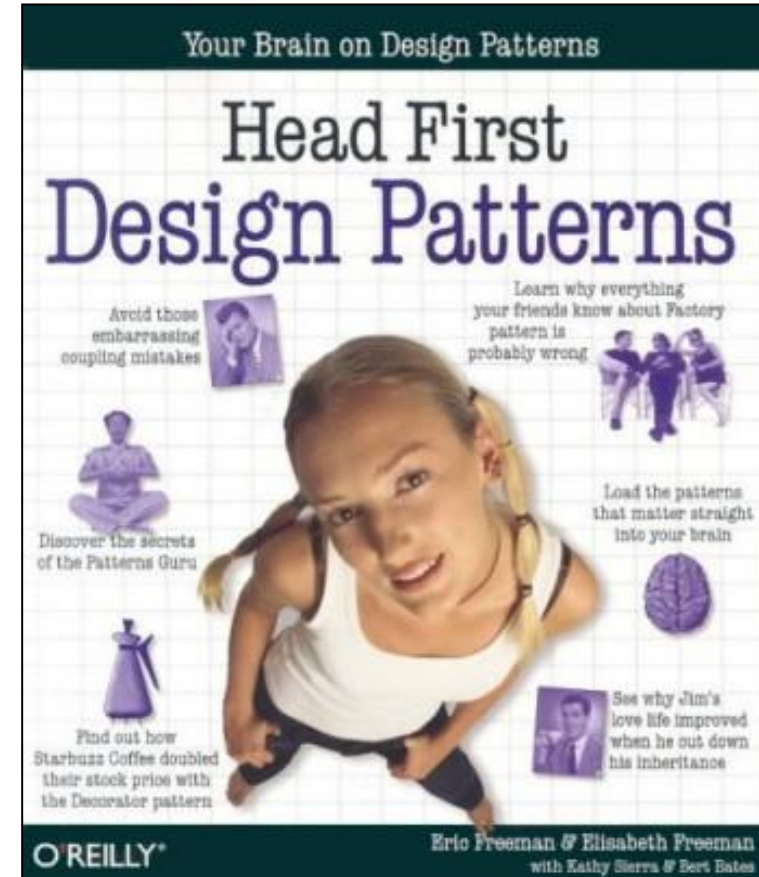
All contents are based on textbooks, research articles, online resources, and personal experience.

Someone has already solved your problems

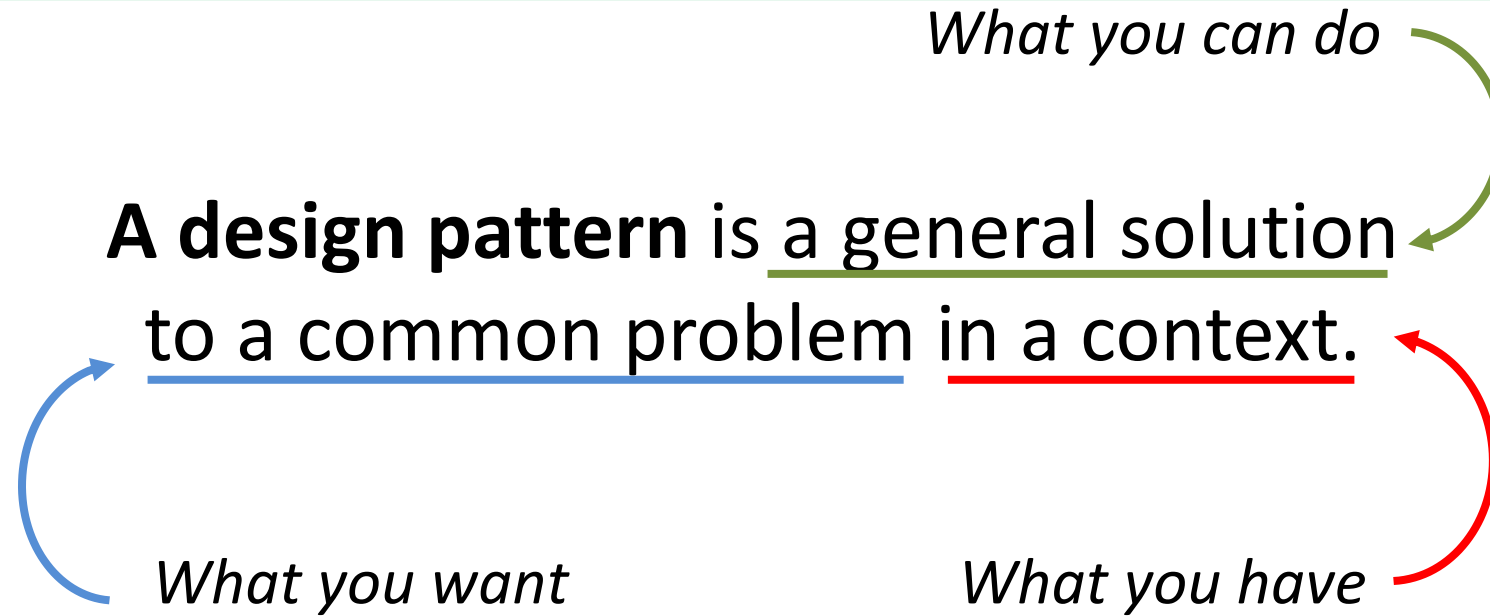
USEFUL BOOKS



The “Gang of Four” book
1994



Head First Design Patterns Book
2004



Each pattern Describes a problem which occurs over and over again in our environment ,and then describes the core of the problem

“a design pattern is a general repeatable solution to a commonly occurring problem in software design”

USAGE OF DESIGN PATTERN



Design Patterns have two main usages in software development:

✓ **Common platform for developers**

- Provide a standard terminology and are specific to particular scenario.

✓ **Best Practices**

- Evolved over a long period of time
- Provide **best solutions** to certain problems faced during software development.
- helps inexperienced developers to learn software design in an easy and faster way.

ELEMENTS OF A PATTERN



- ✓ Four elements to describe a pattern
 - The name of the pattern
 - The purpose of the pattern: what problem it solves
 - How to solve the problem
 - The constraints we have to consider in our solution

TYPES OF DESIGN PATTERNS



There are about 26 Patterns currently discovered ...

These 26 can be classified into 3 types:

1. **Creational:** These patterns are designed for class instantiation. They can be either **class-creation** patterns or object-creational patterns.
2. **Structural:** These patterns are designed with regard to a **class's structure** and composition. The main goal of most of these patterns is to increase the functionality of the class(es) involved, without changing much of its composition.
3. **Behavioral:** These patterns are designed depending on **how one class communicates with others**.

DESIGN PATTERN INDEX



Creational	Structural	Behavioural
Factory Method	Adapter	Template
Abstract Factory	Bridge	Strategy
Builder	Composite	Command
Singleton	Decorator	State
Multiton	Facade	Visitor
Object pool	Flyweight	Chain of Responsibility
Prototype	Front controller	Interpreter
	Proxy	Observer
		Iterator
		Mediator
		Memento

WARNING



ALWAYS GO WITH THE SIMPLEST SOLUTION THAT DOES THE
JOB AND **INTRODUCE PATTERNS ONLY WHERE THE NEED
EMERGES.**

EXCESSIVE USE OF DESIGN PATTERNS CAN MAKE CODE UNNECESSARILY COMPLEX
AND OVER-ENGINEERED.

Strategy Pattern

STRATEGY PATTERN



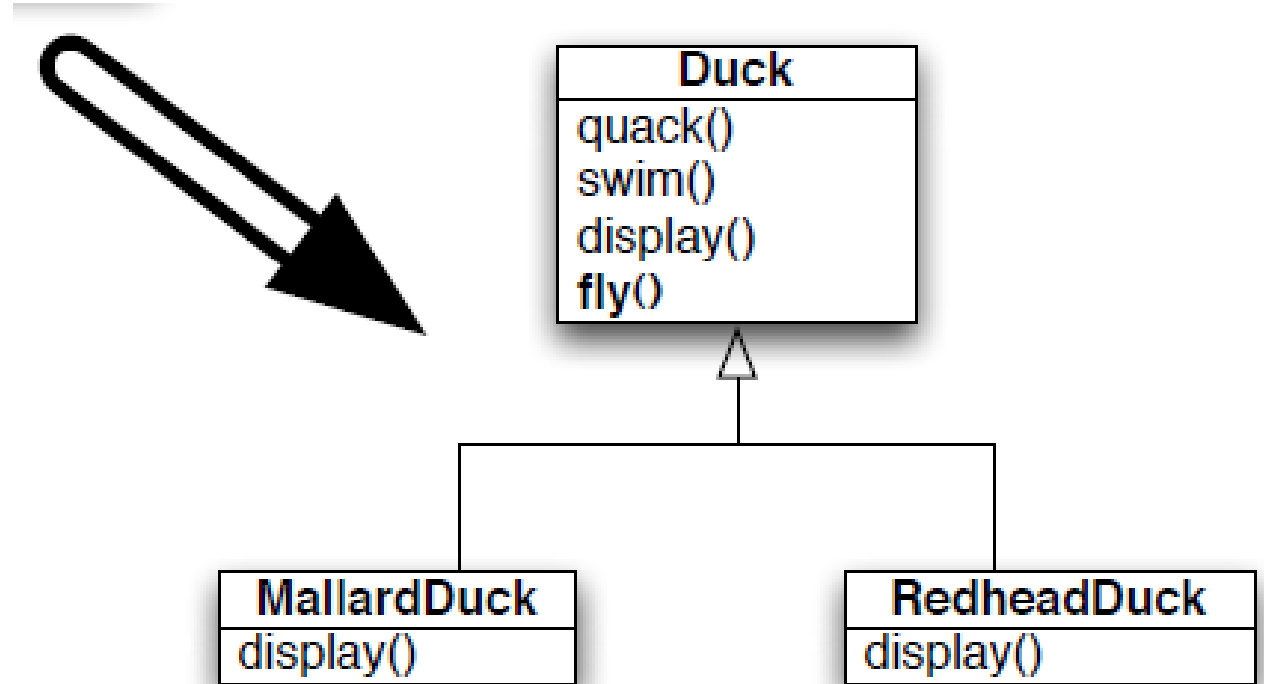
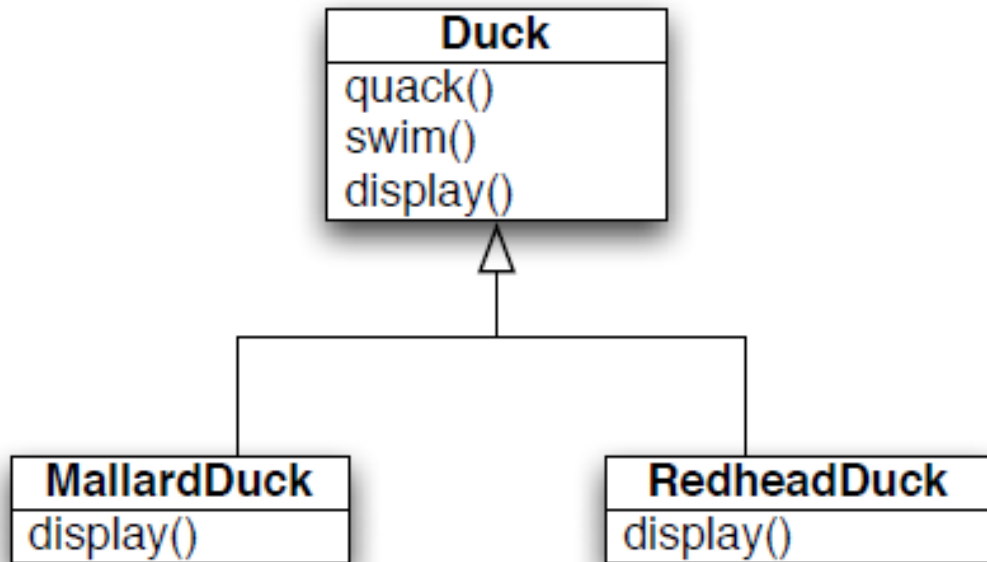
- ✓ it's probably the simplest pattern
- ✓ it's about using composition rather than inheritance
- ✓ it's about understanding that inheritance is not intended for code reuse

DEFINITION



Defines a **family of algorithms**, encapsulates each one, and **makes them interchangeable**. Strategy lets the algorithm vary independently from clients that use it.

STRATEGY PATTERN

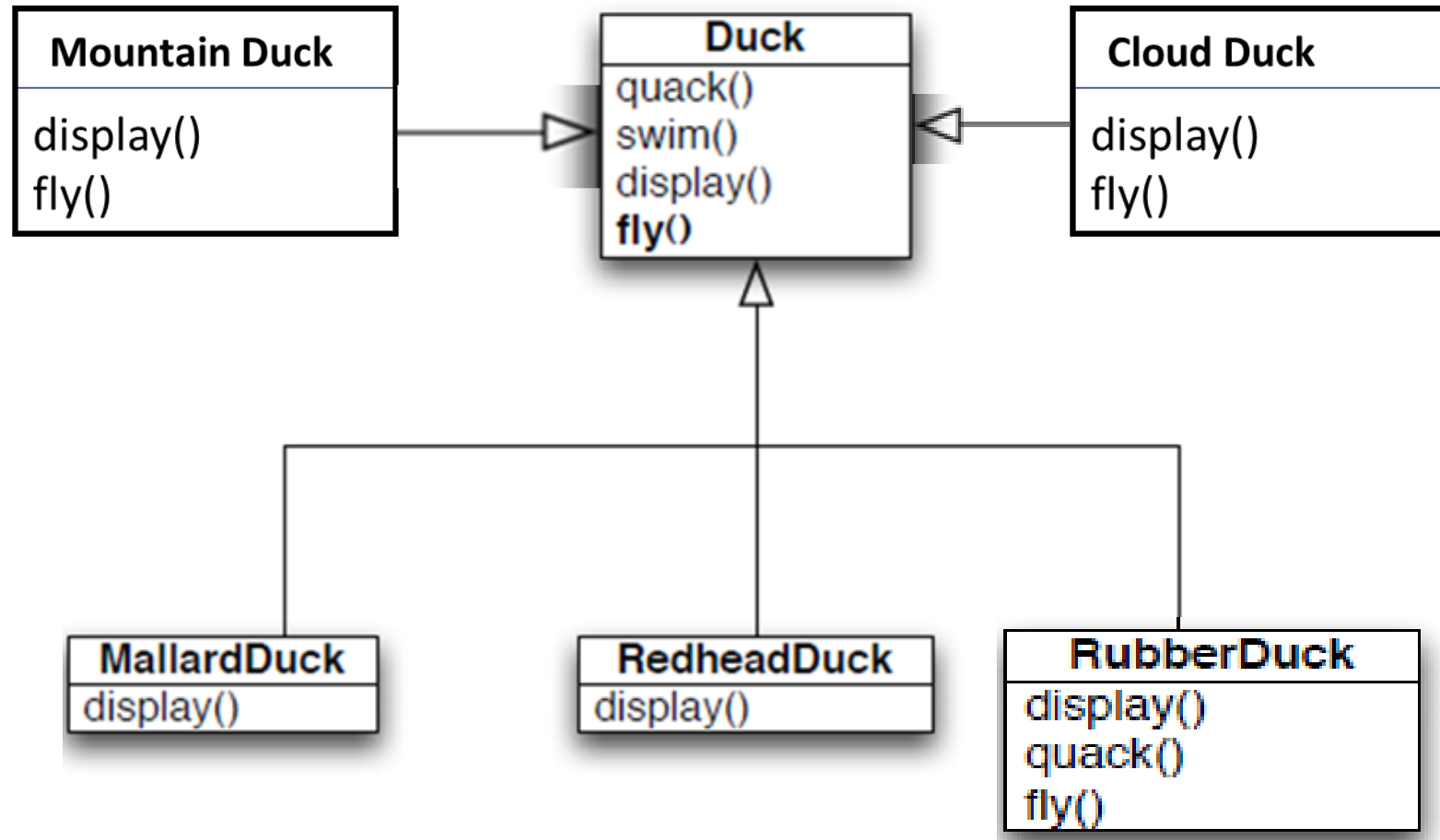


But a request has arrived to allow ducks to also fly.

STRATEGY PATTERN



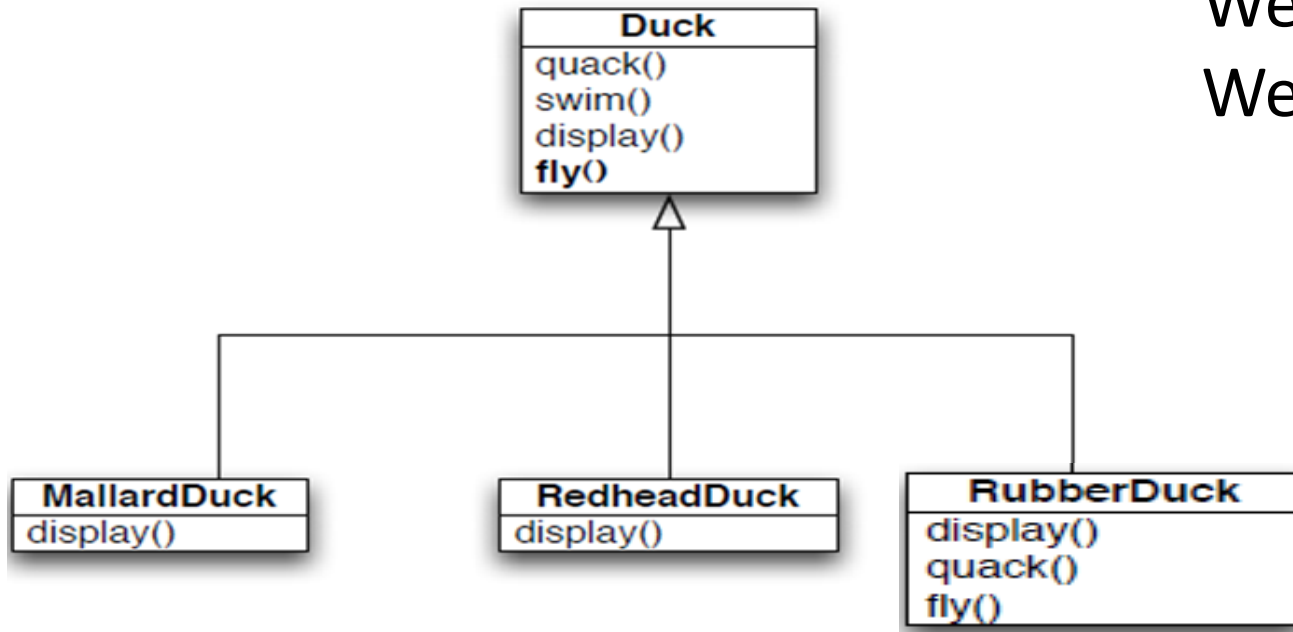
Now we have another duck for example rubber duck which is a fake duck..



Mountain and cloud duck has same flying behavior.

Rubber duck has different flying behavior . We could override `fly()` in **RubberDuck** to make it do Nothing..

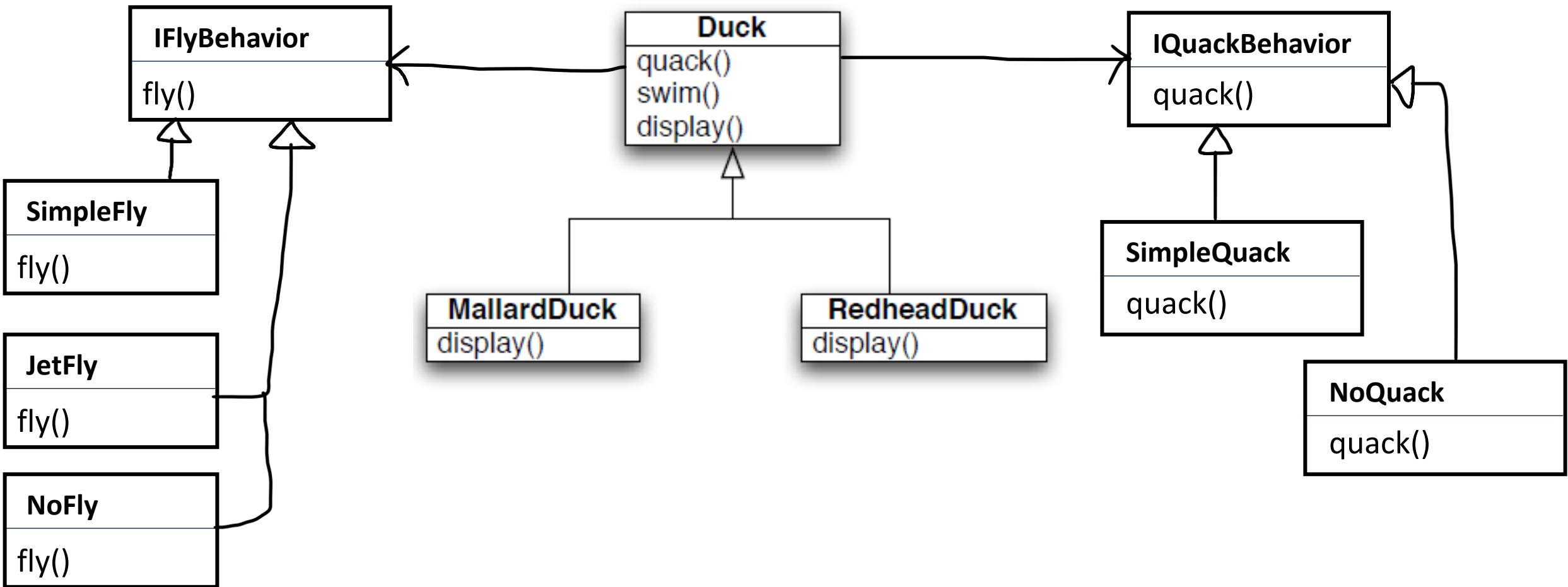
STRATEGY PATTERN



We have algorithm for quacking
We have algorithm for flying...

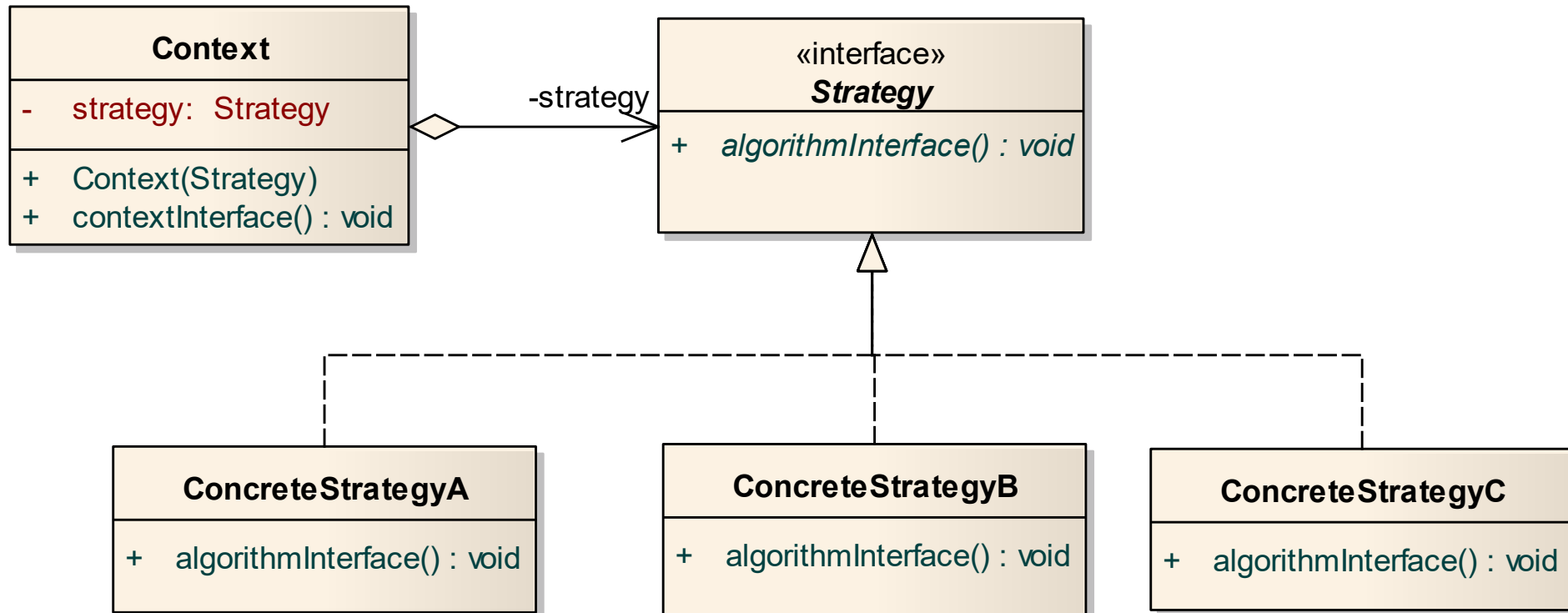
Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

STRATEGY PATTERN



FlyBehavior and QuackBehavior define a set of behaviors that provide behavior to Duck. Duck delegates to each set of behaviors and can switch among them dynamically, if needed.

STRATEGY PATTERN



Observer Pattern

OBSERVER PATTERN



Observer: An object that "watches" the state of another object and takes action when the state changes in some way.

Problem: You have a model object with a complex state, and the state may change throughout the life of your program. You want to update various other parts of the program when the object's state changes.

Solution: Make the complex model object observable.

Observable object: An object that allows observers to examine it (notifies its observers when its state changes).

Permits customizable, extensible event-based behavior for data modeling and graphics.

OBSERVER PATTERN



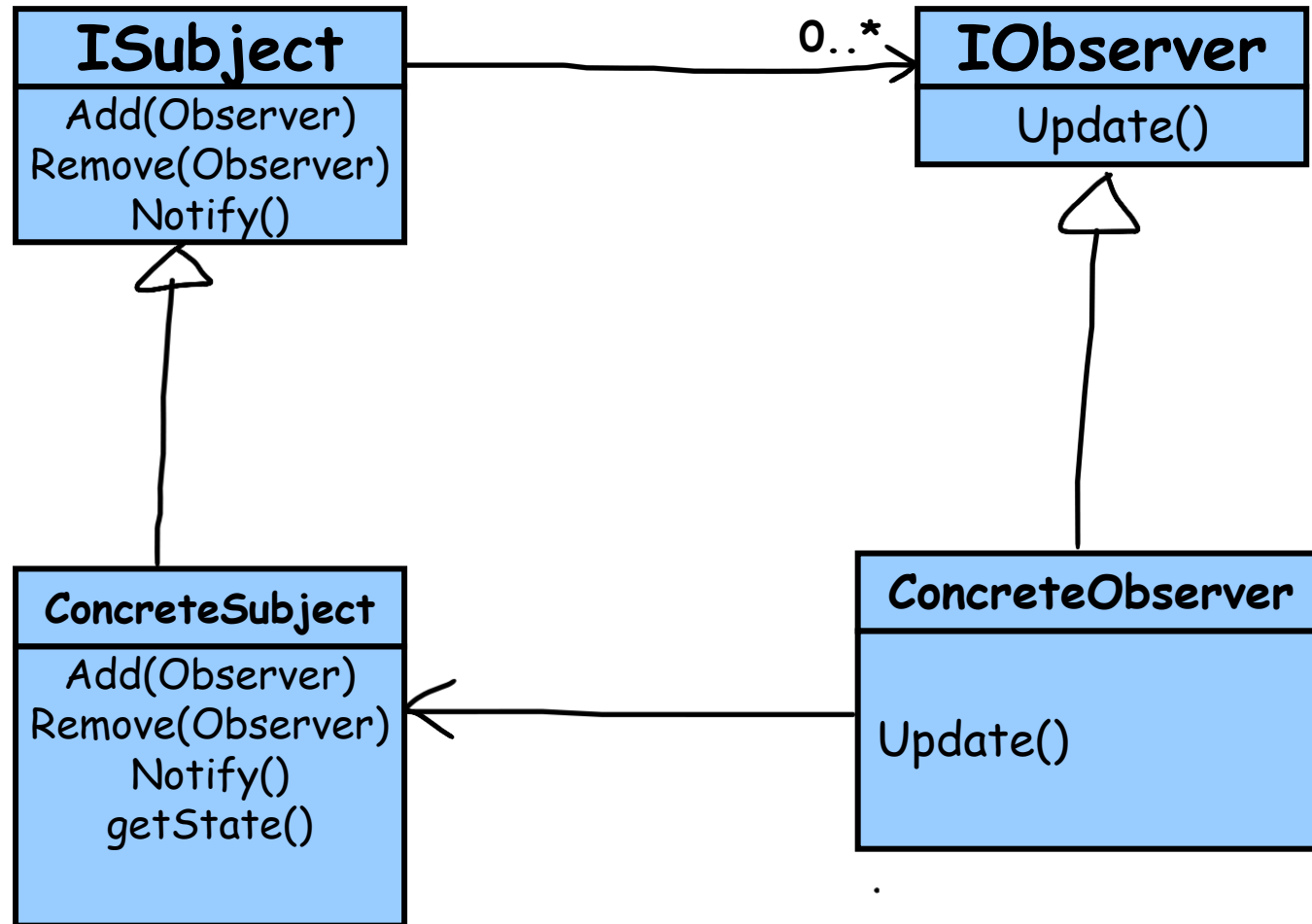
- Subject/observable
 - has a list of observers
 - Interfaces for attaching/detaching an observer
- Observer
 - An updating interface for objects that gets notified of changes in a subject
- ConcreteSubject
 - Stores “state of interest” to observers
 - Sends notification when state changes
- ConcreteObserver
 - Implements updating interface

OBSERVER PATTERN

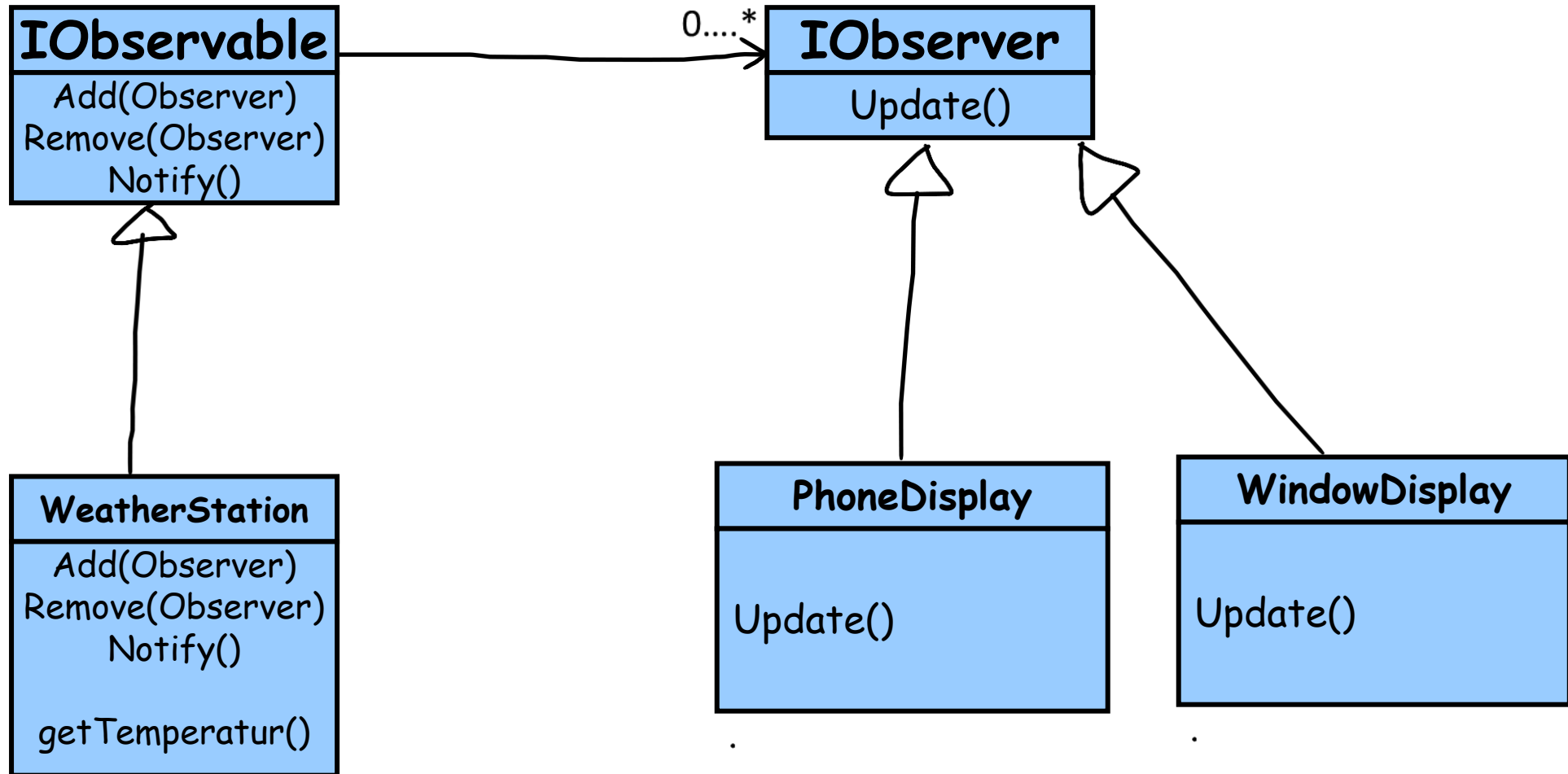


Defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

OBSERVER PATTERN



OBSERVER PATTERN



Decorator Pattern

DECORATOR PATTERN



- ✓ The decorator pattern is a pattern where we write wrapper code to let us extend the core code.
- ✓ We just keep wrapping objects around objects that we want to use so that we keep extending the capabilities of the existing objects by defining new objects that have the capabilities of the existing object.

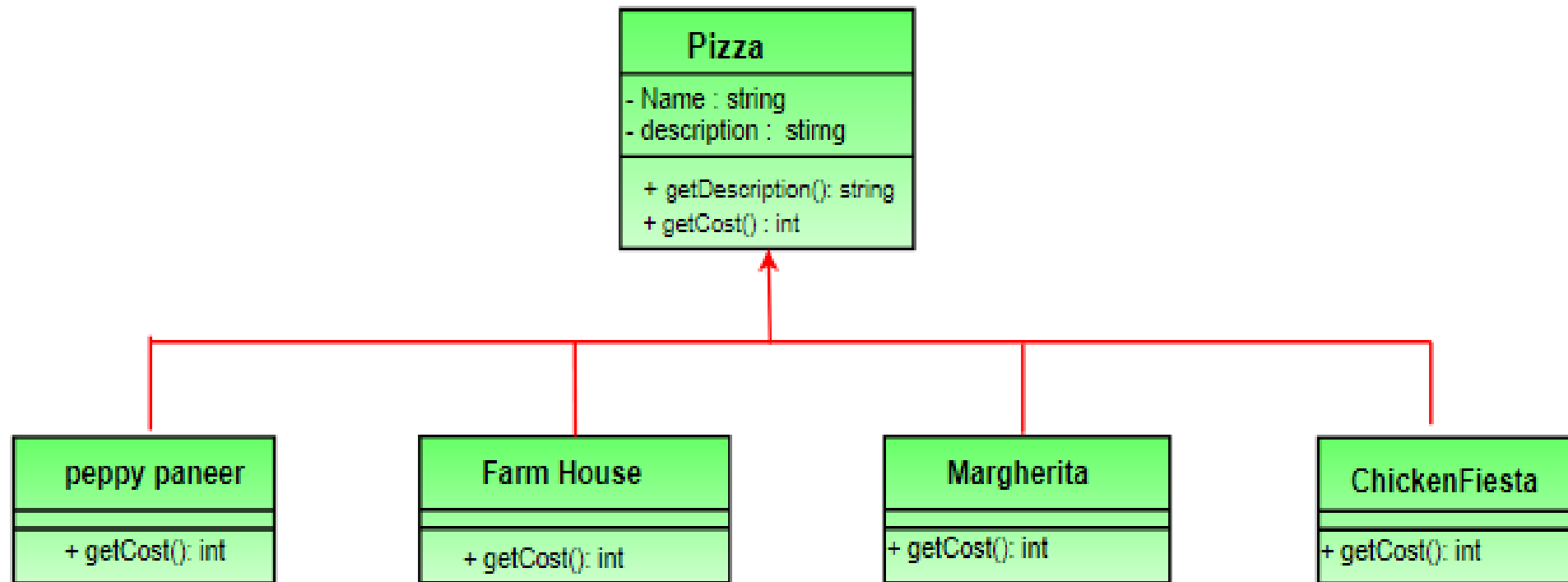
DECORATOR PATTERN



- ✓ how do we accommodate changes in the below classes so that customer can choose pizza with toppings and we get the total cost of pizza and toppings the customer chooses.

Option 1

Create a new subclass for every topping with a pizza.



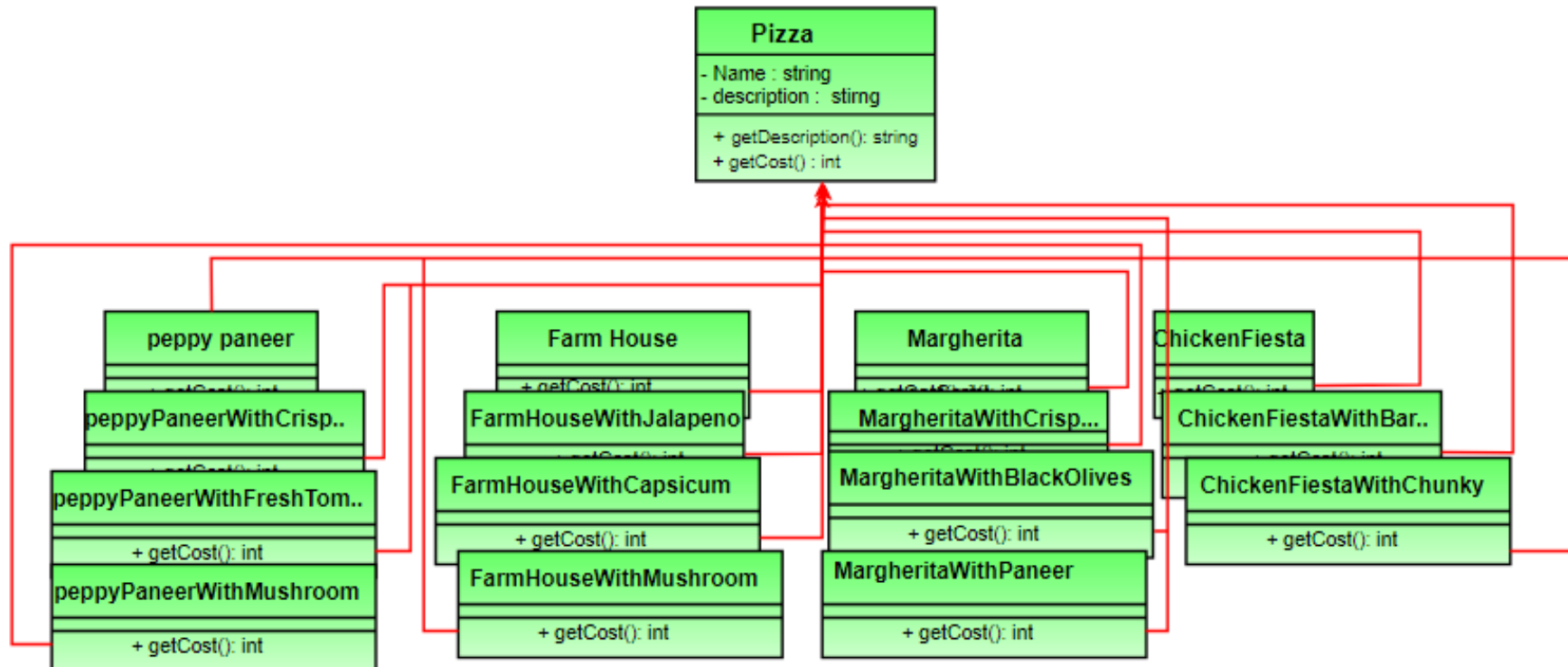
DECORATOR PATTERN



- ✓ how do we accommodate changes in the below classes so that customer can choose pizza with toppings and we get the total cost of pizza and toppings the customer chooses.

Option 1

Create a new subclass for every topping with a pizza.

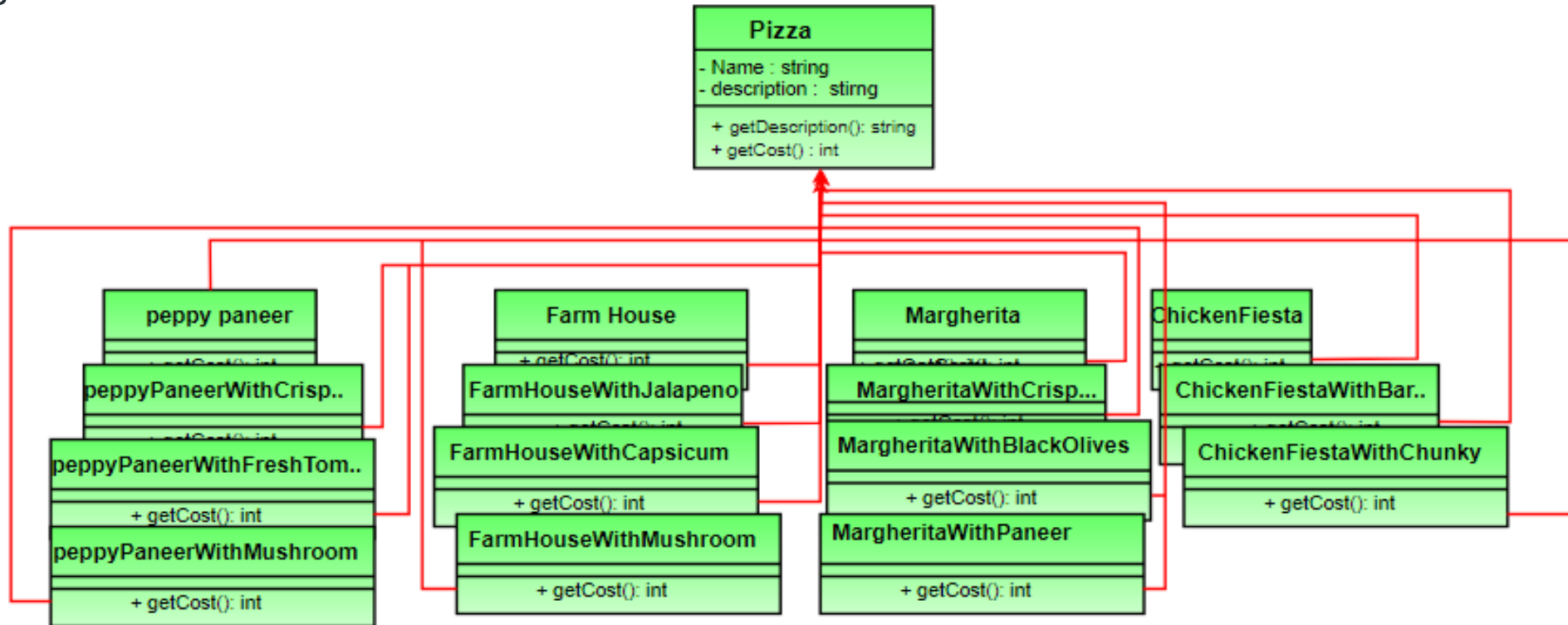


DECORATOR PATTERN



Option 1

- ✓ This looks very complex.
- ✓ There are way too many classes and is a maintenance nightmare.
- ✓ Also if we want to add a new topping or pizza we have to add so many classes. This is obviously very bad design.

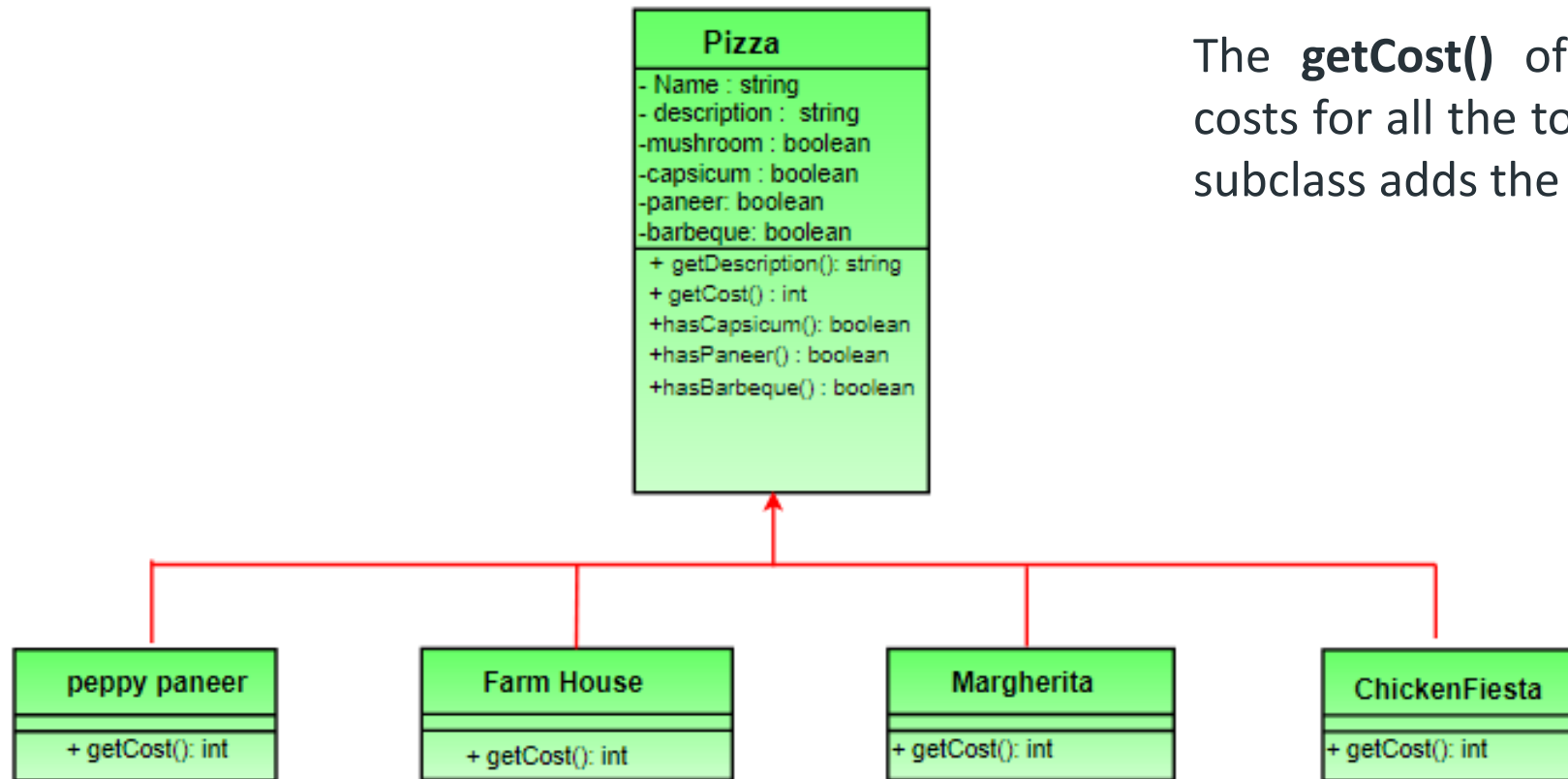


DECORATOR PATTERN



Option 2:

Let's add instance variables to pizza base class to represent whether or not each pizza has a topping.



The **getCost()** of superclass calculates the costs for all the toppings while the one in the subclass adds the cost of that specific pizza.

DECORATOR PATTERN

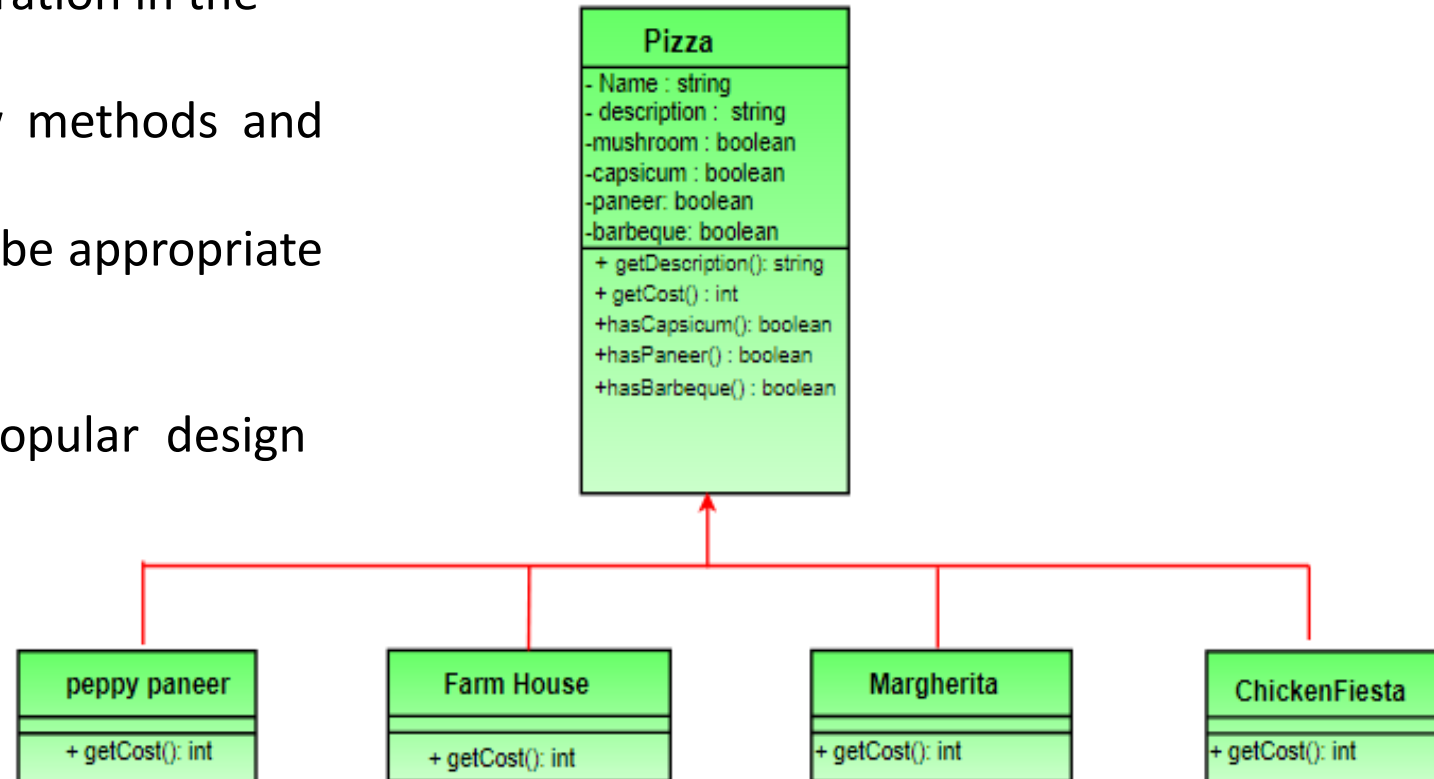


Option 2:

This design looks good at first but problems associated with it are:

- ✓ Price changes in toppings will lead to alteration in the existing code.
- ✓ New toppings will force us to add new methods and alter `getCost()` method in superclass.
- ✓ For some pizzas, some toppings may not be appropriate yet the subclass inherits them.

This design violates one of the most popular design principle – **The Open-Closed Principle**



DECORATOR PATTERN



The decorator pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

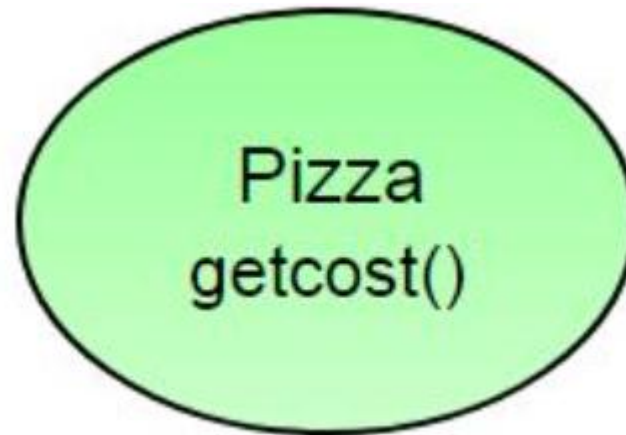
DECORATOR PATTERN



The solution:

- ✓ Previous designs are not a good one
- ✓ Now, we will take a pizza and “decorate” it with toppings at runtime

1.Take a pizza object.



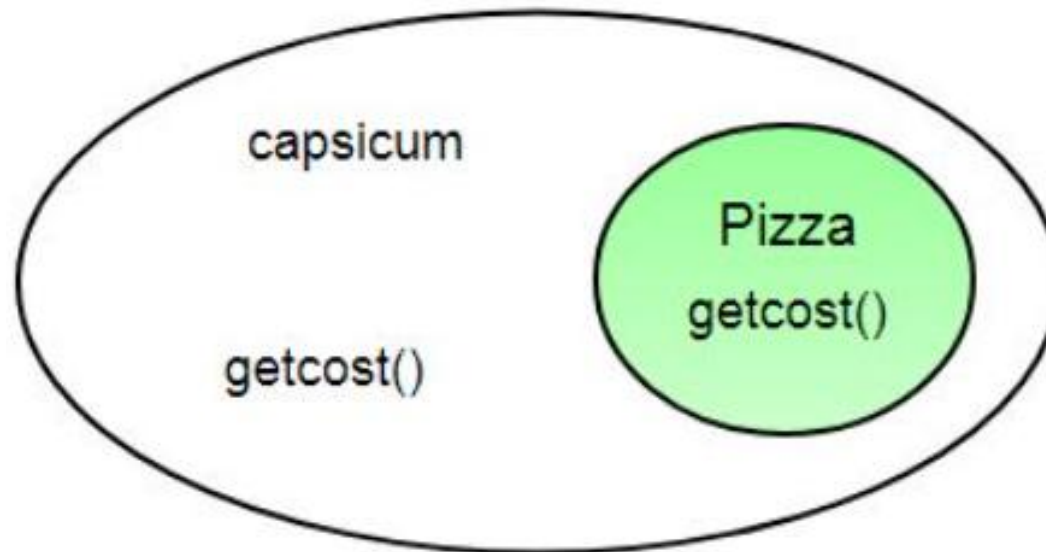
DECORATOR PATTERN



The solution:

- ✓ Previous designs are not a good one
- ✓ Now, we will take a pizza and “decorate” it with toppings at runtime

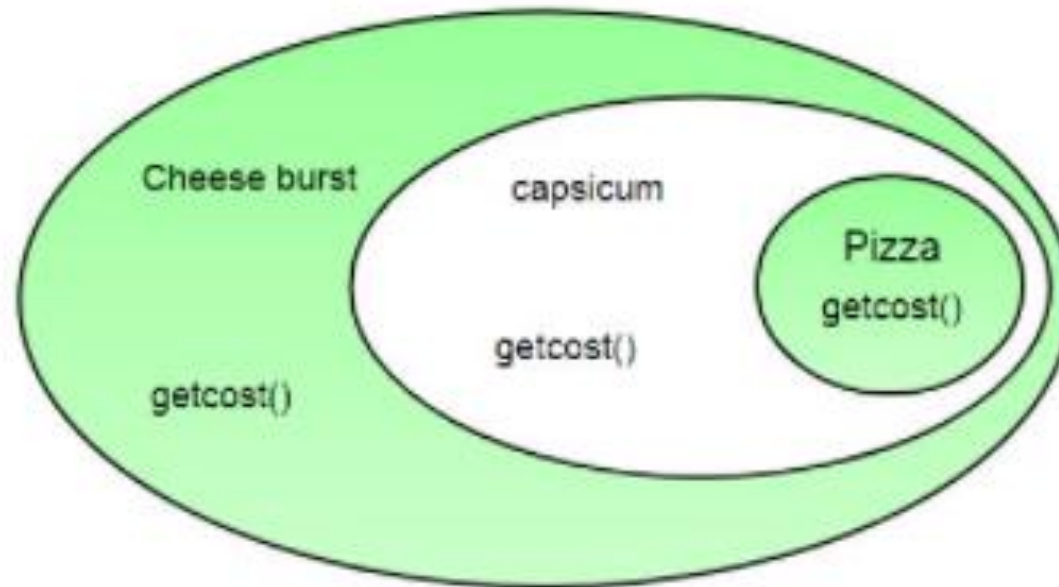
2. “Decorate” it with a Capsicum object.



DECORATOR PATTERN



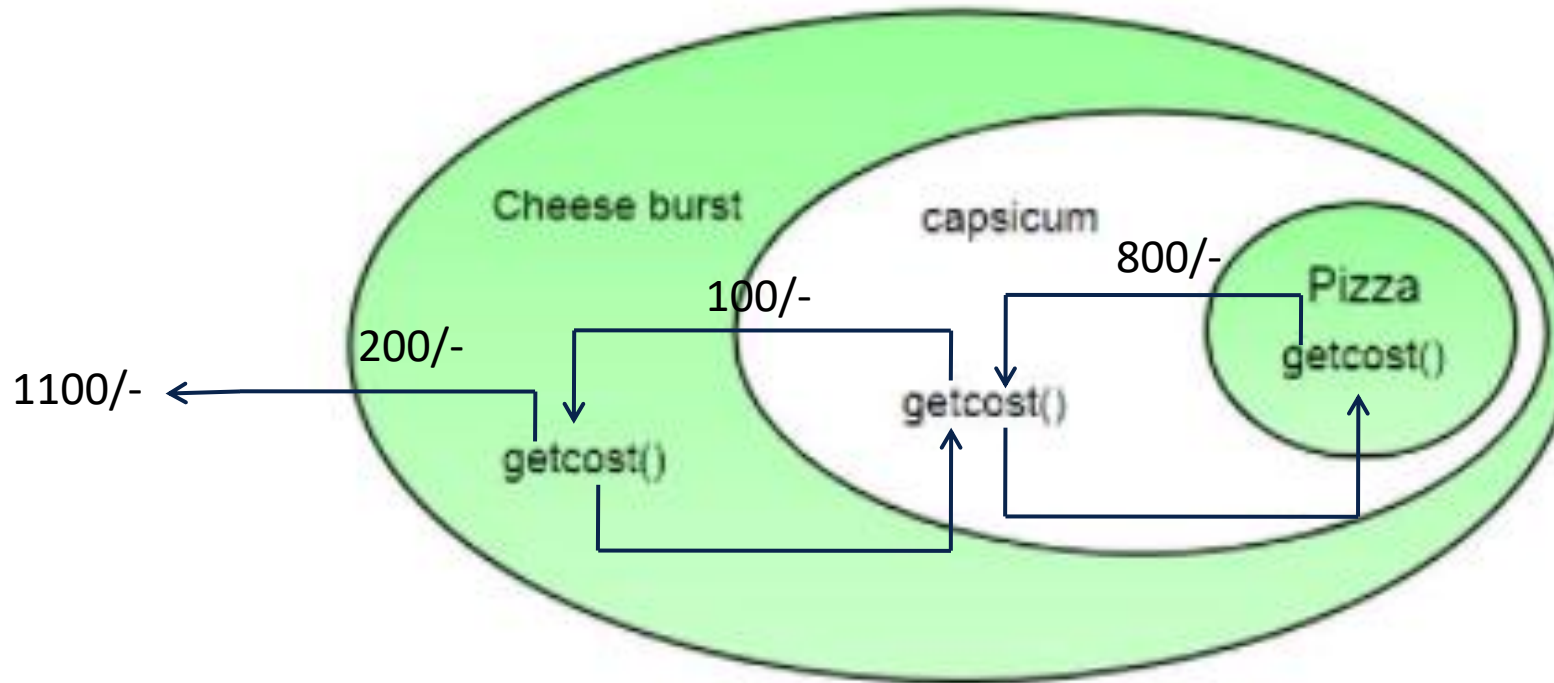
3. “Decorate” it with a CheeseBurst object.



DECORATOR PATTERN



4. Call `getCost()` and use delegation instead of inheritance to calculate the toppings cost.



- ✓ Now we get a pizza with cheeseburst and capsicum toppings.
- ✓ Visualize the “decorator” objects like wrappers.

DECORATOR PATTERN



Here are some of the properties of decorators:

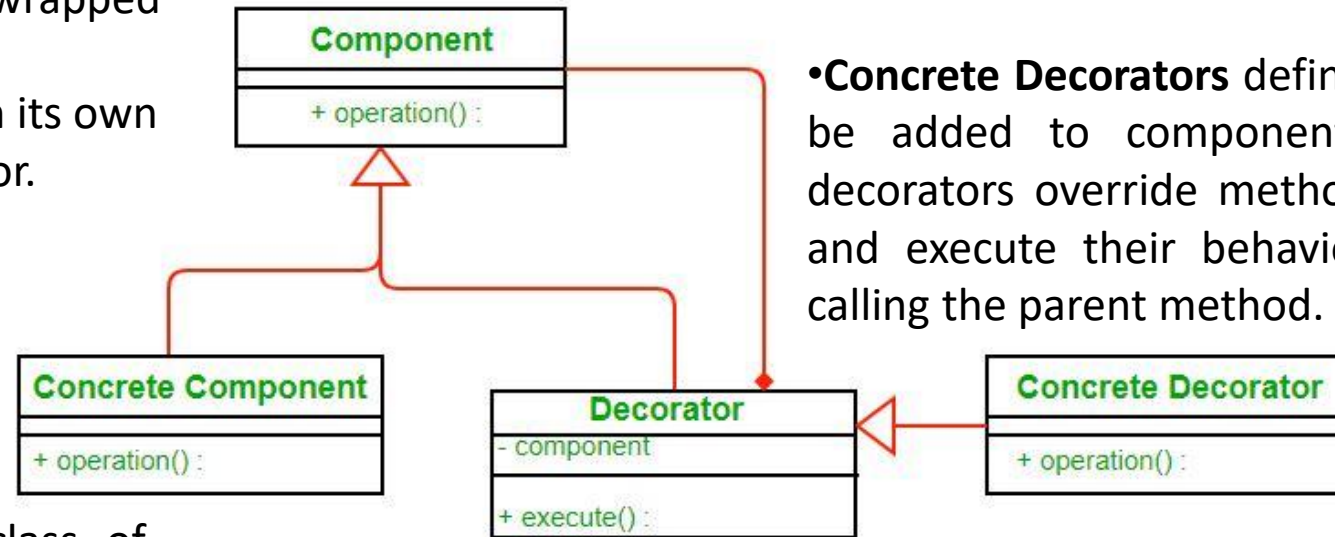
- ✓ Decorators have the same super type as the object they decorate.
- ✓ You can use multiple decorators to wrap an object.
- ✓ We can decorate objects at runtime.

DECORATOR PATTERN



Structure:

- The **Component** declares the common interface for both wrappers and wrapped objects.
- Each component can be used on its own or may be wrapped by a decorator.



- Concrete Decorators** define extra behaviors that can be added to components dynamically. Concrete decorators override methods of the base decorator and execute their behavior either before or after calling the parent method.

- Concrete Component** is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.
- The ConcreteComponent is the object we are going to dynamically decorate.

- The **Decorator** class has a field for referencing a wrapped object. The decorator delegates all operations to the wrapped object.
- Each decorator has an instance variable that holds the reference to component it decorates (HAS-A relationship).

EXAMPLE



Imagine you have a basic interface `Coffee`

```
public interface Coffee {  
    String getDescription();  
    double getCost();  
}
```

Now a `SimpleCoffee` class that implements `Coffee`

```
public class SimpleCoffee implements Coffee {  
    public String getDescription() {  
        return "Simple coffee";  
    }  
    public double getCost() {  
        return 5.00;  
    }  
}
```

How It Works

- The `MilkDecorator` class wraps a `Coffee` object (it has a coffee reference).
- When `getDescription()` or `getCost()` is called on `MilkDecorator`, it calls the same methods on the wrapped coffee object.
- In this way, the decorator can add new functionality (like the addition of "milk") without altering the `SimpleCoffee` class itself.

Now, if we want to add functionality (e.g., add milk or sugar), we can use decorators.

```
public class MilkDecorator implements Coffee {  
    private Coffee coffee; // HAS-A relationship with Coffee  
  
    public MilkDecorator(Coffee coffee) {  
        this.coffee = coffee;  
    }  
  
    @Override  
    public String getDescription() {  
        return coffee.getDescription() + ",  
milk";  
    }  
  
    @Override  
    public double getCost() {  
        return coffee.getCost() + 1.00;  
    }  
}
```

DECORATOR PATTERN



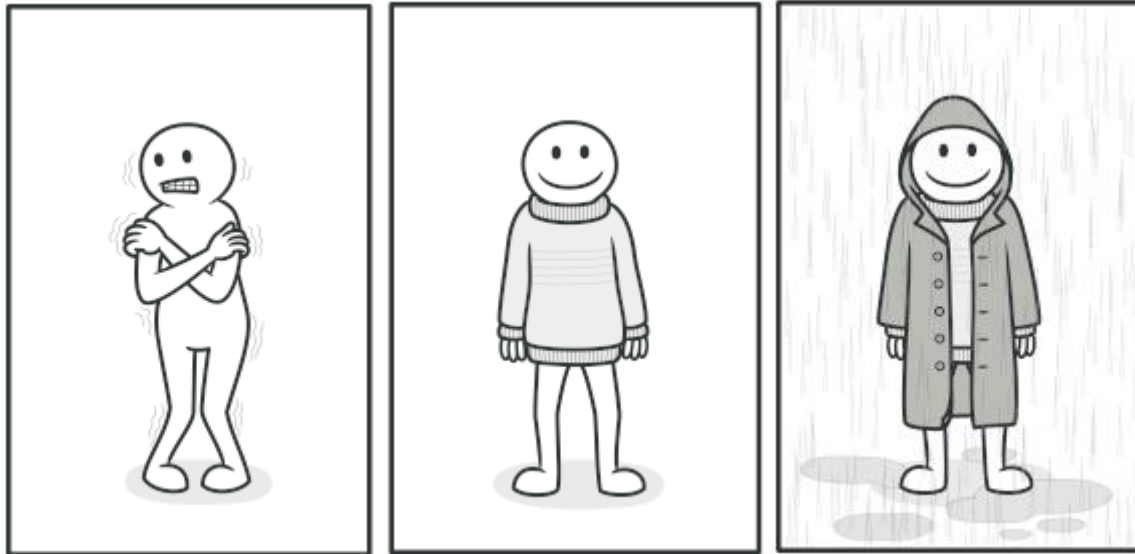
Advantages:

- ✓ The decorator pattern can be used to make it possible to extend (decorate) the functionality of a certain object at runtime.
- ✓ The decorator pattern is an alternative to subclassing. Subclassing adds behavior at compile time, and the change affects all instances of the original class; decorating can provide new behavior at runtime for individual objects.

DECORATOR PATTERN



Real-World Analogy



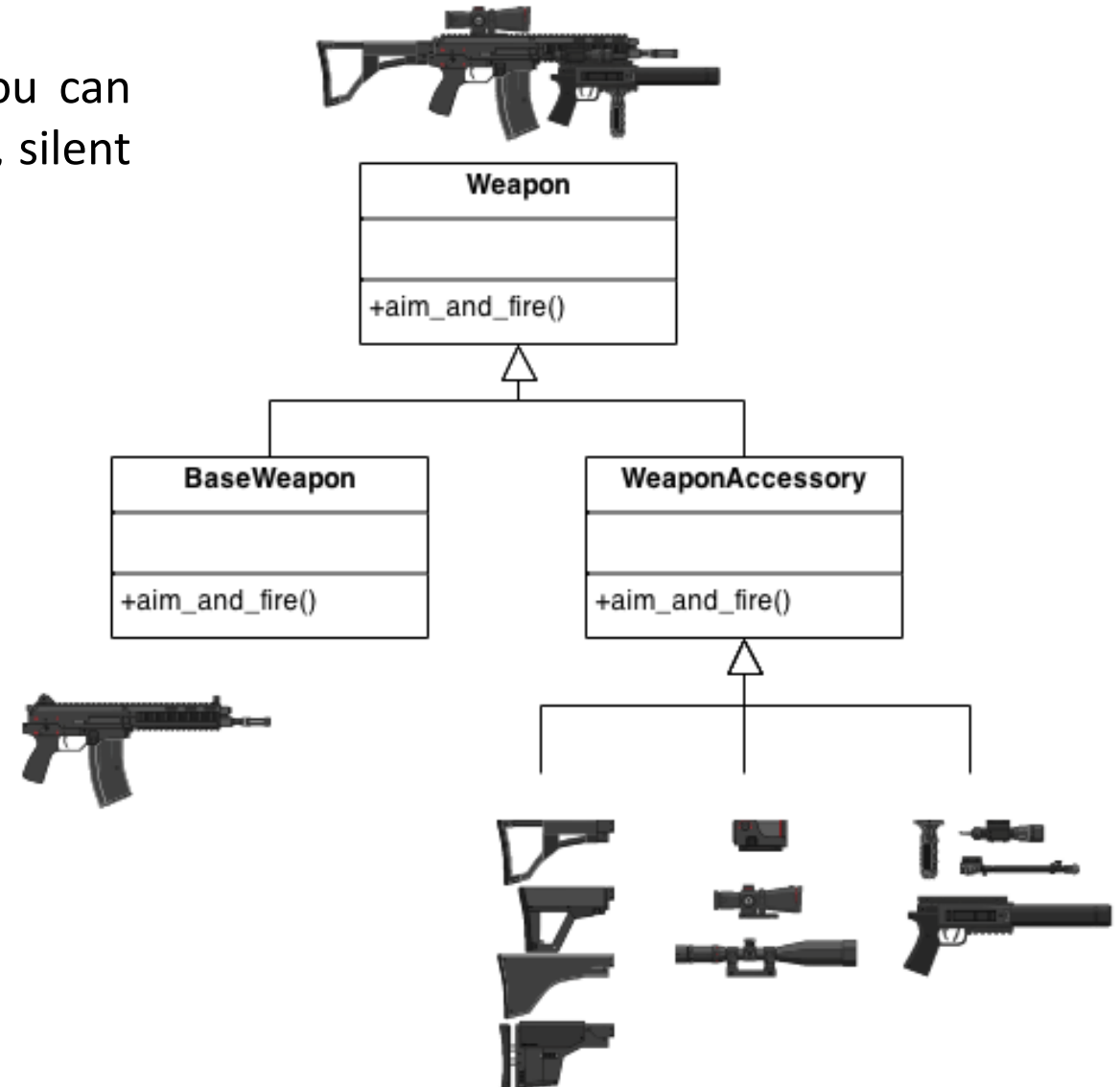
- ✓ Wearing clothes can be an example of using decorators.
- ✓ When you're cold, you wrap yourself in a sweater.
- ✓ If you're still cold with a sweater, you can wear a jacket on top.
- ✓ If it's raining, you can put on a raincoat.
- ✓ All of these garments "extend" your basic behavior but aren't part of you, and you can easily take off any piece of clothing whenever you don't need it.

DECORATOR PATTERN



More scenario:

- ✓ assault gun is a deadly weapon on it's own. But you can apply certain "decorations" to make it more accurate, silent and devastating.



What is the difference between decorator and strategy?
When should we use each of them?